

Esame Software Engineering (AA 2025/26)

08 Luglio 2026 - Lab. Colossus - Via salaria 113

Enrico Tronci

*Computer Science Department, Sapienza University of Rome
Via Salaria 113 - 00198 Roma - Italy*

tronci@di.uniroma1.it

<https://raise.uniroma1.it>

Esercizio 3 (25 punti)

Un sistema di *e-commerce* ci sono P prodotti denotati $\{1, \dots, P\}$ e C customers denotati $\{1, \dots, C\}$. Ogni server offre un solo prodotto e ci sono K servers per lo stesso prodotto. Quindi ci sono KP servers denotati $\{1, \dots, KP\}$. Il server i offre il prodotto $A(i) = (i - 1) \% P + 1$. Quindi $A(1) = 1, \dots, A(P) = P, A(P + 1) = 1, \dots, A(KP) = P$,

Ciascun customer modella un pattern di acquisto per i prodotti. Assumiamo che ciascun customer acquisti sempre V prodotti e sempre nello stesso ordine. I V prodotti che il customer i acquista sono scelti a random tra i P prodotti al momento dell'inizializzazione. L'ordine in cui i prodotti vengono acquistati è l'ordine in cui questi sono estratti a random. Denotiamo con $p(i, j)$ l'indice del prodotto in posizione $j \in \{1, \dots, V\}$ nella lista del customer i .

Il customer i può essere modellato come una catena di Markov con stati $\{0, 1 \dots V\}$. Inizialmente il customer è nello stato 0. Il customer i resta nello stato j per un tempo τ_i , dopo di che transisce nello stato $j + 1$ se $j < V$, altrimenti transisce nello stato 0. Quando il customer transisce dallo stato j allo stato $j + 1$ il customer invia una richiesta di acquisto per il prodotto $p(i, j + 1)$. La richiesta di acquisto viene fatta selezionando a random uno dei K server che dispone del prodotto da acquistare. Cioè uno server u tale che $A(u) = j + 1$.

Il valore τ_i è scelto uniformemente a random nell'intervallo $[a1 + b1 * i, a2 + b2 * i]$, dove $a1, b1, c1, d1$ sono parametri reali. Si noti che in ogni richiesta il customer richiede sempre una solo item del prodotto richiesto. Quindi, per richiedere il prodotto h al server u , il customer invia al server u la richiesta h ed attende dal server la risposta (h) che conferma la spedizione del prodotto richiesto. Se il server u non risponde alla richiesta del customer i entro un certo tempo $\tau(i, u)$ il customer i conclude che il server u è down. In tale situazione il customer ripete il processo di richiesta del prodotto h . Cioè seleziona a random un server che dispone del prodotto ed invia la richiesta. Si noti che il customer non transisce nel nuovo stato ino a quando non ha completato l'acquisto.

Il tempo $\tau(i, u)$ vale $3z$, dove z è estratto uniformemente a random dall'intervallo $[a6 + b6 * s, a7 + b7 * s]$.

Per semplicità si assume che ciascun server ha disponibilità illimitata dei prodotti che può consegnare. Il server può essere in due stati: working, failed. Inizialmente il server è nello stato working. Il server s resta nello stato working

un tempo $W(s)$ dopo di che transisce nello stato failed. Il server s resta nello stato failed per un tempo $F(s)$ dopo di che transisce nello stato working. Il valore $W(s)$ è scelto a random nell'intervallo $[a2 + b2 * s, a3 + b3 * s]$. Il valore $F(s)$ è scelto a random nell'intervallo $[a4 + b4 * s, a5 + b5 * s]$. Nello stato working il server gestisce le richieste dal customer. Quindi se riceve la richiesta del prodotto h dal customer i dopo un tempo $B(s)$ risponde al customer i con il messaggio h . Il valore $B(s)$ è estratto a random nell'intervallo $[a6 + b6 * s, a7 + b7 * s]$. Nello stato failed il server non invia alcun messaggio ed ignora i messaggi ricevuti i quali quindi vengono persi.

Il *tempo di ciclo* per il customer i il valor medio del tempo necessario per completare il ciclo di transizioni che partono dallo stato 0 e ritornano nello stato 0, cioè il tempo medio per completare un ciclo completo di acquisti. La media viene fatta considerando tutti i cicli d'acquisto percorsi nella simulazione fino al tempo t .

Denotiamo con $Q(i, t)$ il *tempo di ciclo* per il customer i fino al tempo t . $Q(i, t)$ viene calcolato come segue. Si supponga che nella simulazione entro il tempo t vengano eseguiti g cicli completi di acquisto e si supponga che il ciclo j richieda un tempo q_j . Allora $Q(i, t) = \frac{1}{g} \sum_{j=1}^g q_j$.

Il *tempo di ciclo* medio globale fino al tempo t $R(t)$ è il valore medio del tempo di ciclo dei customers fino al tempo t . Cioè, $R(t) = \frac{1}{C} \sum_{j=1}^C Q(j, t)$.

1 Obiettivo

Usando M simulazioni Montecarlo Si vogliono riportare in uscita:

- il valore atteso del *tempo di ciclo* medio globale fino alla fine della simulazione, cioè $R(H)$;
- il valore atteso di $Q(i, H)$ per ciascun customer i

2 Parametri

I parametri della simulazione sono:

1. H: orizzonte di simulazione (valore reale positivo)
2. M: numero di simulazioni montecarlo (valore intero positivo)
3. P: numero di prodotti (valore intero positivo)
4. K: numero servers per prodotto (valore intero positivo)
5. C: numero customers (valore intero positivo)
6. V: numero prodotti per customer (valore intero positivo)
7. a1, ... a7: parametri $a1, \dots a7$ del testo (valore reale)

- 8. $b_1, \dots, b_7$: parametri $b_1, \dots, b_7$ del testo (valore reale)
- 9. r : tempo invio messaggi (valore reale positivo)

3 Formato dei parametri di input

I parametri della simulazione sono forniti nel file `parameters.txt`. Le righe del file `parameters.txt` hanno tutte lo stesso formato:

<ID del parametro> <valore del parametro>

Un esempio di file `parameters.txt` è:

```
H 234.6
M 100
P 10
K 3
V 4
C 3
a1 2.1
a2 2.3
a3 2.4
a4 2.5
a5 2.6
a6 3.1
a7 4.1
b1 3.1
b2 3.2
b3 3.3
b4 3.4
b5 3.5
b6 4.1
b7 5.1
r 0.01
```

4 Formato di output

L'output dell'esercizio è memorizzato nel file `results.txt` la cui prima riga è formattata come indicato nelle istruzioni generali.

Le rimanenti righe del file `results.txt` hanno il formato

<ID del parametro> <valore del parametro>

Nel nostro caso:

R < valore medio $R(H)$ >

$Q1$ < valore medio $Q(1, H)$ >

...

$QC < \text{valore medio } Q(C, H) >$

Un esempio di file `results.txt` per il file `parameters.txt` dato sopra è:

```
2025-01-09-Mario-Rossi-1234567
R 12.3
Q1 2
Q2 4
Q3 6
```

5 Struttura Sorgenti

Deve esserci almeno una classe, in un proprio file `cpp`, per ognuno dei processi: `customer`, `server`. Quindi devono esserci almeno i seguenti files: `customer.cpp`, `server.cpp`.